Task Diversity QA

Technical Documentation

Quality assurance for deliveries of human egocentric robotics video — measuring whether a delivery, taken as a whole, is varied enough to train on.

---

1. Introduction

1.1 What this system does

A *delivery* is a batch of egocentric video episodes captured by human operators wearing or holding a recording device. Before such a delivery can be paid for or trained on, one question must be answered about the set itself:

Is the delivery as a whole varied enough? Ten thousand structurally-perfect episodes of one operator doing one task in one warehouse are worthless as training data. Diversity is a property of the *set*, not of any member of it, and it can only be measured in aggregate.

This pipeline answers that question with nine aggregate checks over the delivery's metadata, each producing **PASS / FAIL** against explicit, versioned limits — plus a delivery-composition report that describes the shape of the delivery before any pass/fail judgement.

Component	Answers	Unit of judgement	Output
`diversity_pipeline`	Is the delivery varied enough?	the whole delivery	PASS / FAIL per limit

1.2 Dependencies

The batch CLI is **pure Python 3 standard library**. There is no `requirements.txt` and no `pip install` step. This is deliberate: the pipeline runs in constrained environments — scheduled tasks, minimal containers, analyst laptops — where dependency installation is friction, and the S3 client is small enough to own outright (SigV4 signing is `hmac` plus `hashlib`; transport is `urllib`).

Requirement	Optional?
Python 3.9+ (developed on 3.12)	no
`ffprobe` on `PATH`	**yes** — durations fall back to metadata timestamps, and each affected episode records that it did

2. Architecture

2.1 Shape of the pipeline

```
ingest ──────► normalize ──────► check ──────► report
 (I/O)           (pure)           (pure)         (render)
```

Exactly one module touches the outside world: `ingest.py` for local deliveries, plus `s3client.py` / `s3ingest.py` for the cloud path. Everything downstream — normalization, every check, every threshold comparison, aggregation — is pure over already-loaded data.

This is not architectural decoration. It means every check is testable without a filesystem, a network, or a fixture directory; a delivery can be validated from memory as easily as from disk; and a check can never accidentally depend on I/O ordering, retry behaviour, or wall-clock time.

2.2 Module map — `diversity_pipeline/qa/`

Module	Responsibility
`ingest.py`	**The only local I/O module.** Video/sidecar pairing, natural ordering, `ffprobe` duration measurement
`s3client.py`	Pure-stdlib SigV4 S3 client — list, get, HEAD, presign
`s3ingest.py`	Cloud-direct ingestion; two delivery layouts
`metrics.py`	**All nine diversity checks.** Pure and I/O-free
`config.py`	Every threshold, alias and vocabulary
`profiles.py`	Per-delivery unwrap and confident aliases
`normalize.py`	Flatten to dotted paths; canonical key matching
`models.py`	`Episode` dataclass
`schema.py`	Field registry
`report.py`	JSON / HTML / console rendering with charts
`htmlkit.py`	Charting primitives — donut, hbar, ratio bar, histogram, legend

2.3 The design principles

These are load-bearing. Several checks exist specifically to uphold them, and several *deliberate omissions* exist to avoid violating them.

1 — Never infer, only read. Declared fields are trusted and normalised for *spelling*; nothing is guessed from context. `DIFFICULTY_SYNONYMS` maps `"med"` to `"medium"` because that is a spelling variation of a declared value. It does *not* map task names to difficulties, because that would be invention. An `environment_id` absent from the reference vocabulary is reported as unknown, never resolved to a plausible neighbour — the fix is to add the row to the table, not to guess.

2 — Absent is not passing. A check whose input is missing reports `NOT_COMPUTABLE`, never `PASS`. This is why `business_size` limits report `NOT_COMPUTABLE` on a delivery that lacks the field rather than trivially passing: the distinction between *we checked and it was fine* and *we could not check* is the entire value of the report.

3 — Fail open on ingestion. A malformed sidecar becomes a per-episode finding; the other episodes still load and every other check still runs. An orphan — a video with no sidecar, or a sidecar with no video — is surfaced as an episode carrying a warning and is **never dropped**. An orphan is a finding, not an absence.

4 — Measure; never trust a declaration. Durations are measured from the video artefact wherever possible; metadata timestamps are a claim about it (see §3.3).

5 — Thresholds are data, versioned separately from logic. Every limit lives in `config.py`. Tuning a threshold or moving a field's accepted location is a one-line edit that never touches check logic.

6 — Report checks that pass. Every check emits a record whatever the outcome, so *a check that did not run* stays distinguishable from *a check that passed*. A report with nine explicit statuses is evidence; a report with silence is not.

---

3. Core concepts

3.1 Canonical field resolution

You do not have to match spelling exactly. Before any check runs:

1. Flatten. Nested objects become dotted paths: `data.metadata.task_id`. Only `dict` values are recursed into; lists and scalars are preserved as leaf values. An empty dict is itself a leaf, so its presence is not lost.

2. Canonicalise. Keys are lowercased with `-`, `_`, `.` and whitespace stripped. So `task-id`, `task_id`, `Task_ID` and `taskId` all collapse to `taskid`.

3. Resolve, in this precedence order — full dotted path, then the last (leaf) segment, then any interior segment. Only non-empty values count. An empty string, `null`, or an empty list/object counts as **absent**.

4. Alias priority. Where a record carries two accepted spellings of the same field, the one listed **first in the registry** wins — never whichever key happens to appear first in the JSON. With priority ordering, resolution is deterministic regardless of how a capture app happens to serialise its keys.

3.2 Status model

The pipeline uses a deliberately simple status set, because its checks are aggregate limits rather than per-record conformance:

Status	Meaning	Effect
`PASS`	the limit held	none
`FAIL`	the limit was exceeded	**non-zero exit code**
`WARN`	a target (not a hard limit) fell short, or a passing result is qualified	none
`INFO`	observation, no judgement	none
`NOT_COMPUTABLE`	the required field is absent from the delivery	none

Only `FAIL` sets a non-zero exit code.

3.3 Duration reconciliation

Every diversity limit is denominated in hours, so getting duration right is upstream of every check. Per episode:

Video measured?	Metadata span valid?	Hours used	Source	Warning
yes	yes	**video**	`both`	only if they disagree
yes	no	video	`video`	—
no	yes	metadata span	`metadata`	"no measured video duration"
no	no	**0 h**	`none`	"no duration available"

Where both exist, the measured video wins — it is the artefact, the timestamps are a claim about it. A disagreement is warned about only when it exceeds **1 second absolute** *and* **2 % relative**; below either bound it is encoding jitter, not a defect.

Every episode's duration provenance appears in the report, so a delivery whose hours came mostly from metadata is visibly different from one whose hours were measured.

---

4. Running the pipeline

4.1 Invocation

```
cd diversity_pipeline
python3 run.py <root> [--out OUTDIR] [--profile NAME] [--limit N]
                      [--s3-layout folder|paired] [--factory-dynamic] [--quiet]
```

Argument	Default	Meaning
`root`	*required*	local directory, or `s3://[bucket/]prefix` for cloud-direct QA
`--out`	`qa_out`	output directory
`--profile`	`generic`	source profile — see §6
`--limit N`	none	cap episodes ingested (S3 only, natural order)
`--s3-layout`	`folder`	`folder` = per-episode `<uid>/` folders; `paired` = `videos/` + `metadata/` matched by stem
`--factory-dynamic`	off	use the 2 h dynamic factory cap instead of the 1 h repetitive default
`--quiet`	off	terser console output

Exit codes: `0` all checks clear · `1` any FAIL · `2` usage / ingestion error.

4.2 Ingestion and pairing

Local. Videos (`.mp4` / `.mov`, case-insensitive) are paired with sidecars (`*.meta.json`) within a directory, keyed by `(dir, stem)`. Orphans are surfaced as episodes carrying a warning and are never dropped. Episodes are ordered by the numeric part of the stem, so `video10` sorts after `video2` rather than before it.

Cloud-direct. No download. A single paginated listing groups keys by immediate folder — one request for the delivery rather than one per episode — then metadata is fetched concurrently (24 workers) and the video is referenced by a **presigned URL**, which `ffprobe` reads via HTTP range requests. A full delivery's durations are measured without transferring a single video.

5. The nine checks

Check 1 — Environment category (L1)

Four limits on the distribution of hours across L1 environment categories:

Limit	Threshold
Any single category	≤ **20 %** of hours
Top 5 categories combined	≤ **60 %**
Top 10 categories combined	≤ **85 %**
Distinct categories present	≥ **15**

The report includes the full distribution and, separately, an *environment reference* table listing the 19-entry controlled vocabulary (Retail, Fashion, Repair, Food and beverage, Construction, Food processing, Factory, Hospitality, Automotive, Sports and recreation, Administrative, Healthcare, Cleaning, Laboratory, Agriculture, Beauty and personal care, Entertainment, Laundry, Warehouse) against what the delivery actually reached. Categories never captured are named explicitly — the distinct-count limit is a direct function of that gap — and any `environment_id` *not* in the vocabulary is listed last as unrecognised, never guessed.

Check 2 — Difficulty mix

Shares are of **known-difficulty hours**, not total hours; unknown-difficulty hours are excluded and reported separately.

Limit	Threshold
easy	≤ **30 %**
hard	≥ **5 %**
medium	40 % target — reported as INFO, **not enforced**

Any unknown-difficulty hours downgrade a passing result to `WARN`, so an apparently-good mix computed from a small labelled minority is visibly qualified.

Check 3 — Per-task share

Difficulty	Max share of total hours	Absolute cap
hard	6.0 %	200 h
medium	4.0 %	200 h
easy	1.5 %	200 h

Check 4 — Task × Environment

Difficulty	Cap
hard	40 h
medium	20 h
easy	10 h
factory pair	**1 h** (repetitive, default) or **2 h** with `--factory-dynamic`

A pair is treated as factory when its `environment_l1` contains `factory`, `manufactur` or `industrial`. Factory work is repetitive by nature, so a flat cap replaces the difficulty-based one — an hour of the same assembly-line motion is an hour of the same assembly-line motion regardless of how the task was labelled.

Check 5 — Operator × Task × Environment

Difficulty	Cap
hard	10 h
medium	2 h
easy	1 h

Check 6 — Environment × Operator

Flat cap of **40 h** for any one operator in any one environment.

Check 7 — Business-size caps per environment

Requires `business_size`; `NOT_COMPUTABLE` when the field is absent.

Size	Cap
small	500 h
medium	2 000 h
large	5 000 h

Check 8 — Workforce mix

Requires `worker_type`. By hours: real employees ≥ **70 %**, contractors ≤ **30 %**.

Check 9 — Human–human interaction

Requires the `human_interaction` flag. By **episode count** (not hours): ≥ **5 %** of episodes. This is a target, so falling short is a `WARN`, never a `FAIL`.

Mixed-difficulty groups

Where a group's episodes carry conflicting difficulty labels, the **most severe** label is used (`hard` > `medium` > `easy`) and the conflict count is reported in the check detail. Choosing the strictest interpretation means a mislabelled group is never let through by accident.

6. Source profiles

A profile adapts one delivery's metadata shape without touching the check logic. It declares two things: a pure `unwrap` transform from raw parsed JSON to the metadata object (which only reshapes structure — it never renames keys or invents values), and **confident field aliases** so a canonical field resolves to the delivery's own key.

Only confident mappings belong in a profile. Genuinely-absent fields are left to report honestly.

Profile	Delivery	Adaptation
`generic`	—	none (default)
`xphi-capture-v1`	xphi-capture `processed/`	unwraps array → `data.metadata`; `episode_start_time` → `start_time_unix`, `environment_label` → `environment_l1`
`licensed-egocentric-v1`	licensed egocentric split	`videos/` + `metadata/` layout; `factory_id` → `environment_id`, `worker_id` → `operator_id`
`xphi-episode-v1`	episodic `egocentric_data/`	reads the rich `episode_metadata.json` sidecar rather than the sparse chunk-array `metadata.json`; `environment` → `environment_l1`

`xphi-capture-v1` deliberately does **not** map `task_category`, `device_model`, `device_type`, `software_version` or `camera_configuration` — those mappings would be guesses at semantically different fields. `xphi-episode-v1` deliberately leaves `business_size` unmapped because it is genuinely absent, so that check stays `NOT_COMPUTABLE` rather than being fabricated into a pass.

The discipline that makes profiles safe is refusing to map uncertain fields. A profile that force-maps a semantically-different field to satisfy a check converts an honest `NOT_COMPUTABLE` into a false `PASS`, which is strictly worse than no profile at all.

---

7. Reports and output formats

7.1 Report contents

Beyond the nine check cards, the HTML report opens with a **delivery composition** section that describes the shape of the delivery *before* any pass/fail judgement — every figure derived from the same per-episode rows the checks use:

- Hours by environment level (`environment_l1`, `environment_l2`, `environment_l3` — one chart per level where the delivery declares a hierarchy) and hours by task
- Difficulty mix as an ordinal ratio bar measured against the hard floor — ordered hard→easy, because the rule is a floor on *hard*
- Workforce mix as a ratio bar against the real-employee floor
- Operator concentration (horizontal bars)
- Episode length distribution (histogram, in minutes)
- The environment reference table described in Check 1
- A per-episode duration table with provenance and warnings

7.2 `diversity_report.json`


```
{
  "checks": [
    { "name": "Environment category (L1)", "status": "FAIL",
      "detail": "distinct=3, top-5=100.0%, top-10=100.0%; …",
      "offenders": [ { "label": "warehouse", "hours": 210.4, "pct": 62.1 } ],
      "extra": { "distribution": [ … ] } }
  ],
  "summary": {
    "episodes": 1204, "total_hours": 338.72,
    "status_counts": { "PASS": 2, "FAIL": 4, "NOT_COMPUTABLE": 3 },
    "any_fail": true, "factory_dynamic": false
  },
  "rows": [
    { "stem": "…", "hours": 0.2813, "duration_source": "video",
      "task_id": "…", "operator_id": "…", "environment_id": "…",
      "environment_l1": "…", "difficulty": "medium", "business_size": null,
      "worker_type": "real", "human_interaction": false, "warnings": [] }
  ]
}
```


`rows` is the complete per-episode basis for every aggregate, so any figure in the report can be recomputed or audited independently.

7.3 HTML

The pipeline writes a single self-contained HTML file with inline CSS — no external assets, no network fetches, no build step. It can be emailed, attached to a ticket, or opened from a file:// URL years later and still render. Charts render as inline SVG, each paired with a collapsible table view so the underlying numbers are always one click away.

8. Configuration

Credentials are read from the environment and are **never** hardcoded or committed.

Variable	Default	Purpose
`AWS_ACCESS_KEY_ID`	*required for S3*	access key
`AWS_SECRET_ACCESS_KEY`	*required for S3*	secret key
`S3_ENDPOINT`	*required for S3*	host only, e.g. `sfo3.digitaloceanspaces.com`
`AWS_REGION`	`us-east-1`	e.g. `sfo3`
`AWS_S3_BUCKET_NAME`	—	default bucket when a path omits it

8.1 Tuning thresholds

Every limit is data, not logic.

To change	Edit
a diversity limit	`diversity_pipeline/qa/config.py`
the environment vocabulary	`ENVIRONMENT_NAMES` in the same file
a difficulty / worker-type / business-size spelling	the corresponding `*_SYNONYMS` dict
an accepted field spelling	`DIVERSITY_ALIASES` in `config.py` — **order matters**, first wins

9. Scope boundaries

Things the pipeline reads but deliberately does **not** validate. This section exists because silence must not be mistaken for correctness.

Not checked	Why
Video **content**	never decoded — only duration is measured (via `ffprobe` headers), and only when available
Metadata **truthfulness**	declared task, operator, environment and difficulty labels are trusted after spelling normalisation; verifying them against footage is out of scope
Structural soundness of episodes	that is a per-episode conformance question, answered by a separate validation pipeline, not an aggregate diversity question

10. Extending the system

10.1 Adding a diversity check

1. Write `def check_name(rows: List[DivRow]) -> Dict[str, Any]` in `metrics.py`, returning `_check(name, status, detail, offenders, extra)`.
2. Return `NOT_COMPUTABLE` when the input field is absent — never `PASS`.
3. Put thresholds in `config.py`; add the check to the list in `run()`.

If the check needs a field not yet on `DivRow`, add it to the dataclass, to `DIVERSITY_ALIASES`, and to `build_row`.

10.2 Adding a source profile

Add a `SourceProfile` to `profiles.py` with an `unwrap` (structure only — never renaming keys or inventing values) and *confident* aliases only, then register it in `_REGISTRY`. It becomes available as a `--profile` choice automatically.

10.3 Adding an accepted spelling

One line in `DIVERSITY_ALIASES` (`config.py`). Remember that alias order is priority order: the canonical name goes first, fallbacks last.

Appendix — Check index

#	Name	Requires
1	Environment category (L1)	`environment_l1`
2	Difficulty mix	`task_difficulty`
3	Per-task share	`task_id`
4	Task × Environment	`task_id`, `environment_id`
5	Operator × Task × Environment	`operator_id`, `task_id`, `environment_id`
6	Environment × Operator	`environment_id`, `operator_id`
7	Business-size caps	`business_size`, `environment_id`
8	Workforce mix	`worker_type`
9	Human–human interaction	`human_interaction`